

exp no:14

Timer0 Delay Generation Using AVR C Programming

Aim

To write and execute an AVR C program to toggle all bits of PORTB continuously with a delay generated using Timer0 in Normal Mode without a prescaler.

Theory

Timer0 is an 8-bit timer. In Normal Mode it counts from the value loaded in TCNT0 up to 255 and sets the overflow flag (TOV0) when it rolls over. The program alternately outputs 0x55 and 0xAA on PORTB and uses Timer0 overflow to generate delay. PORTB is configured as an output port. The delay between these outputs is generated using Timer0 overflow. Timer0 starts counting from 0x20 and overflows after 224 counts. When overflow occurs, TOV0 becomes 1 and the delay routine ends. PORTB continuously toggles between 55H (01010101) and AAH (10101010). LEDs connected to PORTB blink alternately.

Algorithm

1. Configure PORTB as output.[Text Wrapping Break]
2. Output 0x55 to PORTB.[Text Wrapping Break]
3. Generate delay using Timer0.[Text Wrapping Break]
4. Output 0xAA to PORTB.[Text Wrapping Break]
5. Generate delay using Timer0.[Text Wrapping Break]
6. Repeat continuously.

Program

```
#include <avr/io.h>
```

```
void T0Delay();
```

```
int main()
```

```
{
```

```
    DDRB = 0xFF;
```

```
    while(1)
```

```
    {
```

```
        PORTB = 0x55;
```

```
        T0Delay();
```

```
        PORTB = 0xAA;
```

```
        T0Delay();
```

```
    }
```

```
}
```

```
void T0Delay()
```

```
{
```

```
    TCNT0 = 0x20;
```

```
    TCCR0 = 0x01;
```

```
    while((TIFR & 0x01)==0);
```

```
    TCCR0 = 0x00;
```

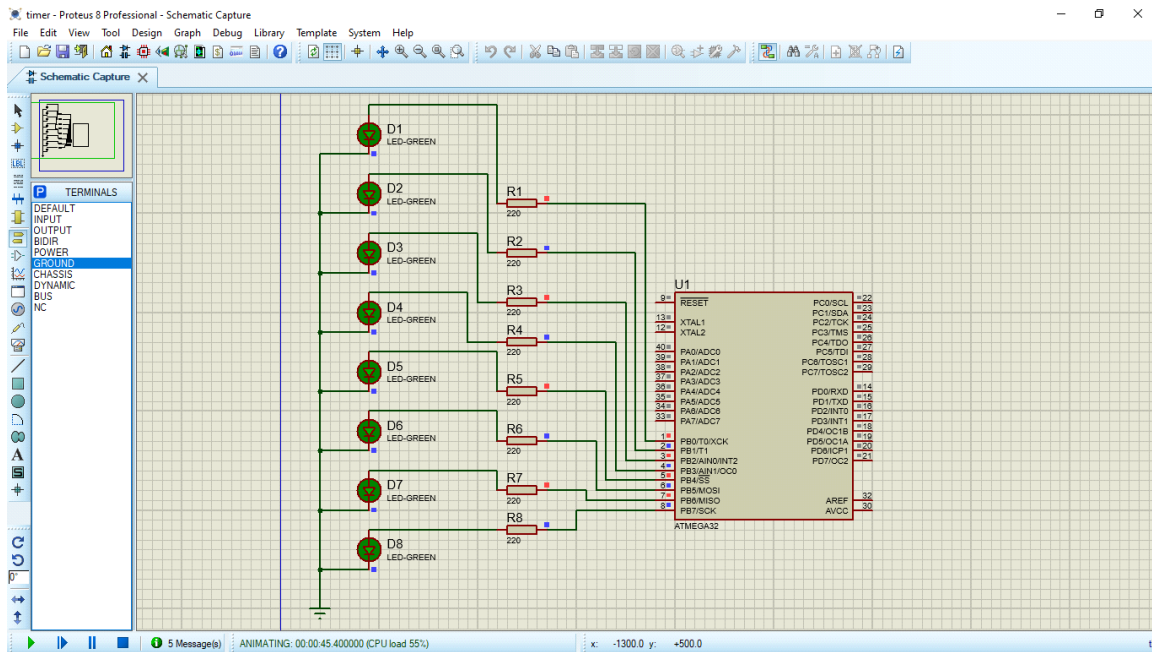
```
    TIFR = 0x01;
```

```
}
```

Result

Thus, the AVR C program was successfully executed to toggle PORTB using Timer0 in Normal Mode without a prescaler.

figure:



exp no: 15

program:

```
#include <avr/io.h>

// Function prototype for delay
void T0Delay();

int main(){

    DDRB = 0xFF; // Configure PORTB as output (all 8 pins)

    while (1){    // Infinite loop

        PORTB = 0x55; // Output pattern 01010101 on PORTB

        T0Delay();    // Call software delay

        PORTB = 0xAA; // Output pattern 10101010 on PORTB

        T0Delay();    // Call software delay

    }

}

void T0Delay(){

    // Simple software delay using a for loop

    // 'volatile' prevents compiler optimization of the loop

    for (volatile unsigned long i = 0; i < 50000; i++){

        // Empty loop body

    }

}
```

exp no: 16

program:

```
#include <avr/io.h>

#include <util/delay.h> // Include built-in delay functions

// Main program

int main(){

    DDRB = 0xFF; // Configure PORTB as output (all 8 pins)

    while (1){ // Infinite loop

        PORTB = 0x55; // Output pattern 01010101 on PORTB

        _delay_ms(500); // Delay for 500 milliseconds

        PORTB = 0xAA; // Output pattern 10101010 on PORTB

        _delay_ms(500); // Delay for 500 milliseconds

    }

}
```

exp no: 17

Experiment: Toggle PORTB using Timer0 in CTC Mode (ATmega32)

Aim

To toggle all bits of PORTB continuously using Timer0 in CTC mode with no prescaler.

Theory

In CTC (Clear Timer on Compare Match) mode, Timer0 counts from TCNT0 to the value in OCR0. When TCNT0

equals OCR0, the OCF0 flag is set and the counter is cleared automatically. The program polls OCF0 to generate a

delay before toggling PORTB.

Algorithm

1. Configure PORTB as output.
2. Output 0x55.
3. Call delay routine.
4. Output 0xAA.
5. Call delay routine.
6. Repeat forever.

Program

```
#include <avr/io.h>

// Function prototype for delay

void T0Delay(void);

int main(void){

    DDRB = 0xFF; // Configure PORTB as output (all 8 pins)

    while (1){

        PORTB = 0x55; // Output pattern 01010101 on PORTB

        T0Delay(); // Call delay function

        PORTB = 0xAA; // Output pattern 10101010 on PORTB

        T0Delay(); // Call delay function

    }

}

void T0Delay(void){

    TCNT0 = 0x00; // Initialize Timer0 counter to 0

    OCR0 = 0xC0; // Set Output Compare Register (compare value = 192)

    TCCR0 = 0x09; // Start Timer0 in CTC mode (WGM01=1) with prescaler = 1

    while ((TIFR & 0x02) == 0); // Wait until OCF0 (Output Compare Flag) is set

    TCCR0 = 0x00; // Stop Timer0

    TIFR = 0x02; // Clear OCF0 flag by writing 1 to it

}
```

Program Explanation

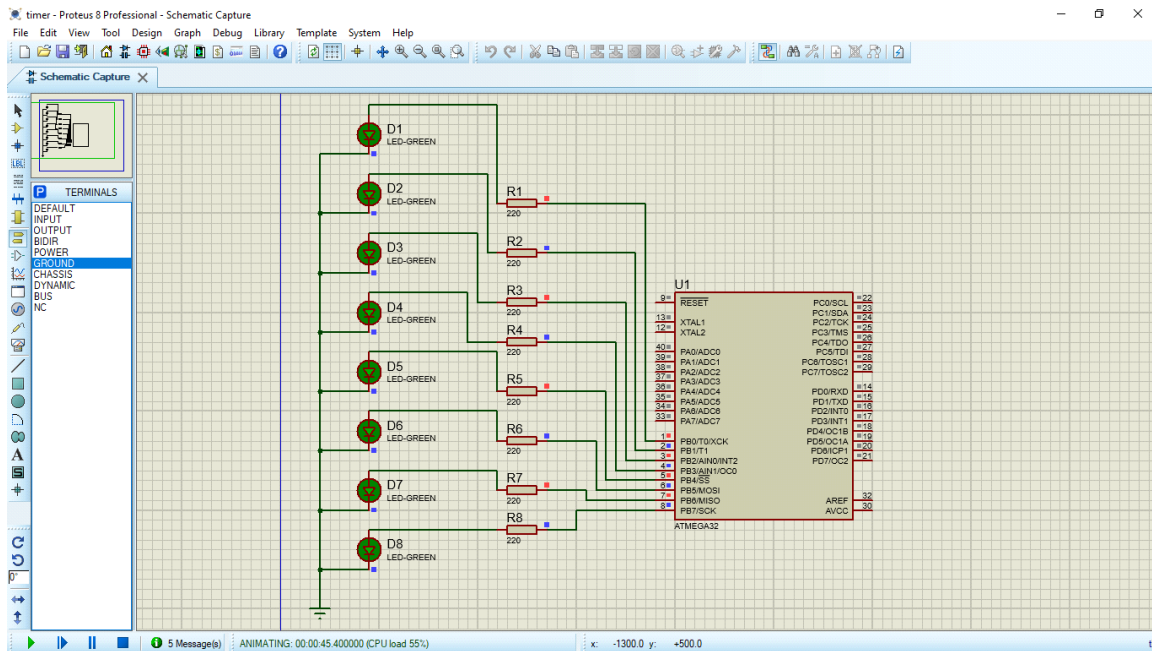
DDRB configures PORTB as output. OCR0 stores the compare value. TCCR0=0x09 selects CTC mode with no

prescaler. The program waits until OCF0 is set, stops the timer, clears OCF0, and toggles PORTB.

Result

The LEDs connected to PORTB alternately display 0x55 and 0xAA with a delay generated by Timer0 in CTC mode.

figure:



exp no: 18

Expt: - TIMERO AS COUNTER

Aim:

Design and implement an AVR program using ATmega32 where Timer0 is configured in counter mode to count external pulses applied to the T0 pin (PD4). The current 8-bit count value must be continuously displayed on PORTB using LEDs.

Objective:

To understand and implement Timer0 as a hardware counter for counting external events using AVR microcontroller.

THEORY:

AVR microcontrollers like ATmega32 provide Timer/Counters that can operate in both time and event counting modes. A timer uses the internal clock, while a counter uses external pulses.

- Timer0 is an 8-bit timer/counter in ATmega32.
- It can count external events applied to the T0 pin (PD4) when configured in counter mode.
- The counting occurs on rising edges if configured with no prescaler.
- The value in TCNT0 register represents the current count.
- By displaying this value on an output port (e.g., PORTB), one can observe the external events being counted in real time.

1. Counter Mode Configuration Steps:

- Configure PD4 (T0) as input.
- Configure PORTB as output to display the counter value.
- Set TCCR0 to count external clock on rising edge without prescaler.

- Read TCNT0 continuously and send the value to PORTB.

This mode is helpful in event counting applications like:

- Pulse counting from sensors
- Rotational speed measurement
- Frequency counting

PROCEDURE:

1. Develop the program in Atmel Studio and build the hex file.
2. Design the circuit in Proteus with:
 - Pulse generator connected to PD4 (T0)
- 8 LEDs connected to PORTB
3. Load the hex file into the ATmega32 microcontroller.
4. Run the simulation or flash to hardware.
5. Observe the LED pattern changing as pulses are received.

PROGRAM:

```
#define F_CPU 1000000UL

#include <avr/io.h>

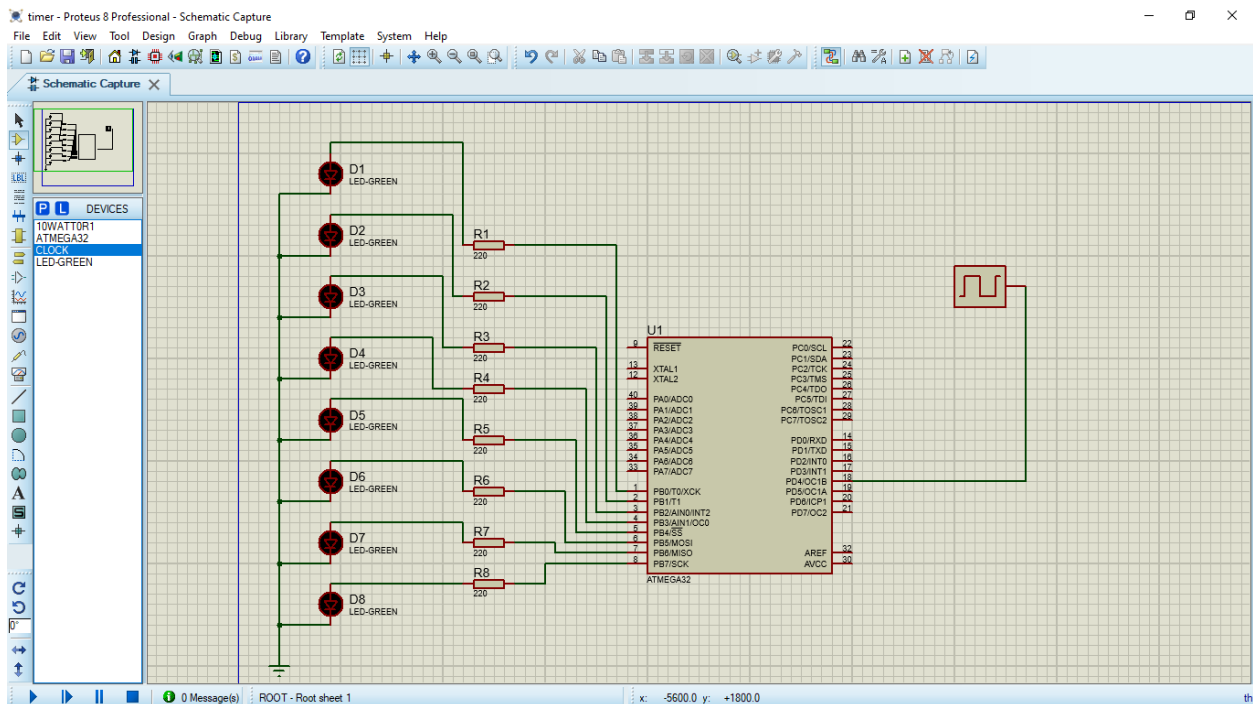
void main(void)
{
    DDRB = 0xFF;           // Set PORTB as output (to display count)
    DDRD &= ~(1 << PD4);   // Set PD4 (T0) as input for external pulses
    TCCR0 = 6;             // Timer0: External clock source on T0, falling edge
    TCNT0 = 0;             // Clear Timer/Counter0

    while (1){
        PORTB = TCNT0;     // Output current count to PORTB
    }
}
```

RESULT:

The AVR C program for Timer0 in counter mode was developed and compiled successfully using Atmel Studio. The hex code was loaded into Proteus simulation environment. The simulation displayed the count value on PORTB, incrementing with each external pulse received at pin PD4 (T0). This experiment successfully demonstrated external event counting using Timer0.

figure:



exp no: 19

AVR Timer0 as 16-Bit External Counter (ATmega32)

Aim

To extend the 8-bit Timer0 into a 16-bit counter using the Timer0 overflow flag (TOV0) and display the count on PORTC and PORTD.

Program

```
#include <avr/io.h>

int main(void){

    DDRC = 0xFF; // Configure PORTC as output (lower 8 bits)
    DDRD = 0xFF; // Configure PORTD as output (upper 8 bits)
    PORTB = 0x01; // Enable pull-up on PB0 (T0 pin for external clock input)
    TCNT0 = 0x00; // Clear Timer0 counter
    TCCR0 = 0x06; // Configure Timer0: external clock source on T0 pin (falling edge)
    PORTD = 0x00; // Initialize upper byte to 0
    while (1)    // Infinite loop
    {
        PORTC = TCNT0; // Display lower byte of counter on PORTC
        if (TIFR & (1 << TOV0)) // Check if Timer0 overflow flag is set
        {
            TIFR = (1 << TOV0); // Clear overflow flag by writing 1
            PORTD++;           // Increment upper byte (overflow counter)
        }
    }
}
```

Working

Timer0 counts external pulses applied to the T0 pin. The current 8-bit count is displayed on PORTC. Whenever Timer0 overflows from 255 to 0, the TOV0 flag is set. The program clears this flag and increments PORTD. Thus, PORTD acts as the upper 8 bits while PORTC acts as the lower 8 bits, together forming a 16-bit counter (PORTD:PORTC) with a counting range of 0 to 65535.

Explanation:

Statement	Explanation
PORTD = 0x00;	Initializes PORTD to zero. PORTD acts as the upper 8 bits of the 16-bit counter.
while(1)	Creates an infinite loop so the program runs continuously.
PORTC = TCNT0;	Copies the current Timer0 count value to PORTC. PORTC represents the lower 8 bits of the counter.
if(TIFR & (1<<TOV0))	Checks whether Timer0 has overflowed (counted from 255 back to 0).
TIFR = (1<<TOV0);	Clears the Timer0 Overflow Flag (TOV0) by writing logic 1 to it.
PORTD++;	Increments PORTD whenever an overflow occurs, forming the upper byte of a 16-bit count.

Working Principle

Timer0 starts counting from 0 to 255.

The current count is displayed on PORTC.

When Timer0 reaches 255 and overflows to 0, the overflow flag (TOV0) is set.

The program detects the overflow and increments PORTD.

PORTC = Lower 8 bits (LSB)

PORTD = Upper 8 bits (MSB)

Together, PORTD and PORTC form a 16-bit counter ranging from 0 to 65535.

Example:

PORTD (MSB)	PORTC (LSB)	Decimal Count
0	0	0
0	100	100
0	255	255
1	0	256
1	1	257
255	255	65535

Result

A 16-bit up-counter is implemented using 8-bit Timer0.

- PORTC displays the lower byte.
- PORTD displays the upper byte.
- Maximum count achieved = 65535 (0xFFFF).

Figure:

